Design and Verification of a UART Transmitter and Receiver Using Verilog and Questa Sim

Name: Nour Ahmed Mohamed Ibrahim

Table of Contents

Abstract

This project presents the design, implementation, integration, and functional verification of a Universal Asynchronous Receiver/Transmitter (UART) system using Verilog Hardware Description Language (HDL). The developed system consists of an 8-bit UART transmitter, an 8-bit UART receiver, and a top-level integration module connecting both units through a serial communication line. The transmitter converts parallel data into a serial UART frame consisting of a start bit, eight data bits, an optional parity bit, and a stop bit. The receiver performs the reverse operation by detecting the incoming frame, sampling the serial data using 8× oversampling and three-sample majority voting, checking the start, parity, and stop conditions, and reconstructing the original 8-bit data. Both even and odd parity configurations are supported, as well as operation without parity. The complete design was verified using a Verilog testbench with a 200 MHz system clock and simulated using Questa Sim. Multiple data patterns and parity configurations were tested to verify correct transmission, reception, data integrity, timing, and continuous frame operation.

Introduction

Digital systems commonly require reliable communication between different processing units, peripherals, and electronic devices. Several serial communication protocols are widely used for this purpose, including UART, SPI, and I2C. Among these protocols, UART is one of the simplest and most widely used methods for asynchronous serial communication.

Universal Asynchronous Receiver/Transmitter (UART) is a hardware communication interface that converts parallel digital data into a serial stream for transmission and converts received serial data back into parallel data. UART is described as asynchronous because the transmitter and receiver do not share a dedicated clock signal. Instead, both devices operate according to an agreed timing configuration.

UART communication is also full duplex, meaning that transmission and reception can occur independently and simultaneously. A transmitter places serialized information on the communication line, while a receiver monitors the line and reconstructs the transmitted information.

In this project, a complete UART communication system was developed using Verilog HDL. The system includes both the transmitter and receiver, allowing the transmitted serial data to be directly connected to the receiver for loopback verification.

UART Communication Principle

A UART communication frame is transmitted sequentially over a single serial data line. When there is no communication, the serial line remains at logic 1, which is referred to as the idle state. When transmission begins, the transmitter first drives the line to logic 0, creating the start bit. This transition informs the receiver that a new frame is beginning. The start bit is followed by eight data bits. In this implementation, the data is transmitted LSB first, meaning that the least significant bit is transmitted before the remaining bits. Depending on the configuration, a parity bit may then be transmitted. The parity bit provides a basic error-detection mechanism. The design supports both even parity and odd parity. Finally, the transmitter sends a stop bit, which is logic 1. After the stop bit, the communication line returns to the idle state. Therefore, the frame structure implemented in this project is:

- Start Bit → 8 Data Bits → Optional Parity Bit → Stop Bit

For a frame without parity:

- Start Bit → 8 Data Bits → Stop Bit

Project Objectives and Specifications

The main objective of this project is to design and verify a complete UART transmitter and receiver according to the provided specifications. The major design requirements are:

- Implement a UART transmitter using Verilog HDL.
- Implement a UART receiver using Verilog HDL.
- Support 8-bit parallel data.
- Support optional parity.
- Support both even and odd parity.
- Transmit data LSB first.
- Keep the serial line high during the idle condition.
- Prevent the transmitter from accepting new data while it is busy.

- Implement 8x oversampling in the receiver.
- Use majority voting to improve receiver sampling reliability.
- Detect invalid start, parity, and stop conditions.
- Assert the receiver data-valid signal only for correctly received frames.
- Support consecutive UART frames.
- Use an asynchronous active-low reset.
- Verify the complete system using a 200 MHz clock in QuestaSim.

System Architecture

The complete project is divided into three major sections:

1. **UART Transmitter**

Responsible for converting the 8-bit parallel input data into a serial UART frame.

2. **UART Receiver**

Responsible for detecting, sampling, checking, and reconstructing the received serial frame.

3. **UART System Top Module**

Connects the transmitter output directly to the receiver input to create a complete UART loopback system.

The overall communication path is:

Parallel Data → UART TX → Serial Communication Line → UART RX → Parallel Data

This architecture allows the complete communication process to be verified within a single simulation environment.

UART Transmitter Design

The UART transmitter is responsible for taking an 8-bit parallel data word and transmitting it serially according to the UART frame format. The transmitter is divided into five functional blocks:

1. Clock Divider
2. Parity Calculator
3. Serializer
4. TX Finite State Machine
5. TX Multiplexer

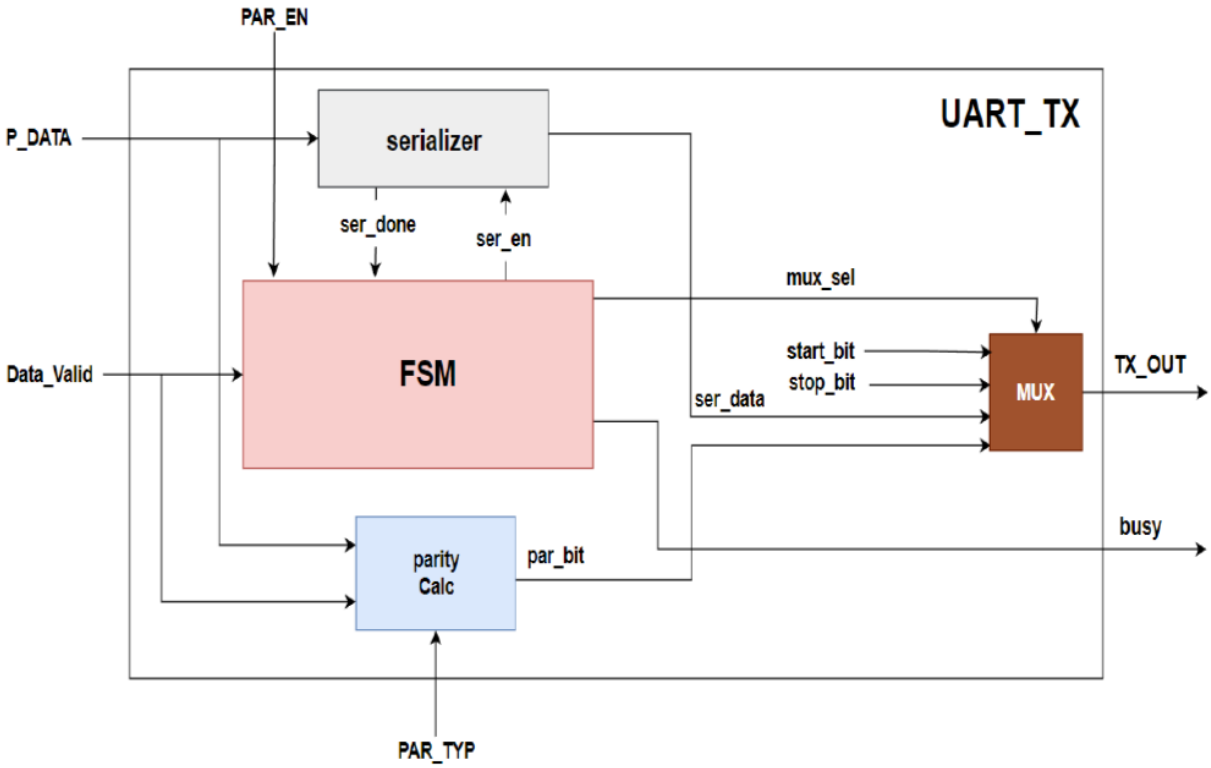

Figure 1: TX Block Diagram

1. **Parity Calculator**

The parity calculator generates the parity bit from the eight input data bits. The design supports two configurations:

- **PAR_TYP = 0:** Even parity
- **PAR_TYP = 1:** Odd parity

The parity calculation is performed when valid data is accepted by the transmitter. The resulting parity bit is stored and later inserted into the UART frame.

Why is this block required?

Parity provides a simple mechanism for detecting certain transmission errors. The receiver performs a corresponding parity calculation and compares the expected parity with the received parity bit.

2. **Serializer**

The serializer converts the 8-bit parallel data into a serial sequence. When a valid data word is accepted, it is loaded into an internal shift register. The serializer then shifts the data one position at a time so that each bit can be transmitted individually. The implementation uses LSB-first transmission, meaning that bit 0 is transmitted first. A bit counter keeps track of the number of transmitted data bits and indicates when all eight bits have been processed.

Why is this block required?

UART communication is serial, while the input data is provided in parallel. The serializer provides the necessary conversion from parallel representation to serial transmission.

3. TX Finite State Machine

The transmitter FSM is the main control unit of the UART transmitter. It determines which part of the frame should be transmitted at each stage. The FSM consists of the following logical states:

IDLE → START → DATA → PARITY → STOP → IDLE

When the transmitter is idle, it waits for a valid data request. Once data is accepted, it enters the START state and transmits logic 0. The FSM then enters the DATA state and controls the serializer while the eight data bits are transmitted. If parity is enabled, the FSM enters the PARITY state. Otherwise, it directly enters the STOP state. After transmitting the stop bit, the FSM returns to IDLE. The FSM also generates the busy signal to indicate that the transmitter is currently processing a frame.

Why is this block required?

The UART frame must be transmitted in a strict sequence. The FSM provides an organized method of controlling this sequence and prevents the different frame components from being transmitted at the wrong time.

4. TX Multiplexer

The TX multiplexer selects the appropriate signal to drive the serial output. Depending on the transmitter FSM state, it selects:

- Start bit
- Serialized data
- Parity bit
- Stop bit

This allows all parts of the UART frame to be combined onto a single serial output line.

Why is this block required?

A UART uses one serial output line. The multiplexer provides a controlled method for selecting the correct frame component at each stage of transmission.

UART Receiver Design

The UART receiver performs the reverse operation of the transmitter. It receives the serial UART stream and reconstructs the original eight-bit data. Unlike the transmitter, the receiver must determine the correct sampling point because there is no shared clock. The receiver therefore uses 8x oversampling and samples the incoming signal around the center of each bit. The receiver contains the following functional blocks:

1. Edge and Bit Counter
2. Data Sampling
3. Deserializer
4. Start Check
5. Parity Check
6. Stop Check
7. RX Finite State Machine

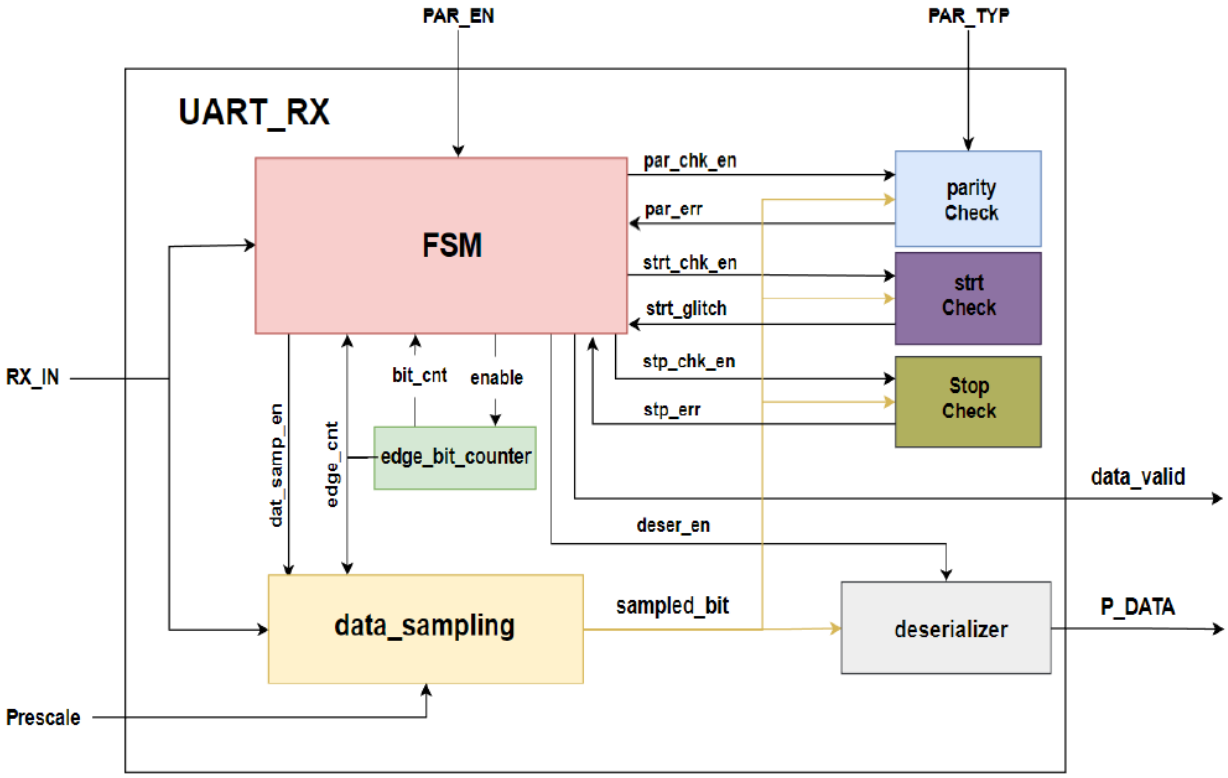

Figure 2: RX Block Diagram

1. Edge and Bit Counter

The receiver requires timing information to determine where each incoming UART bit should be sampled. The edge counter tracks the receiver's position within the current UART bit period. The bit counter tracks the number of data bits that have already been received. Since the receiver operates with 8× oversampling, the edge counter progresses through the oversampling positions of each bit.

Why is this block required?

The receiver has no clock signal from the transmitter. The counters therefore provide the timing reference required to identify the center of each received bit.

2. **Data Sampling**

The data sampling block is one of the most important parts of the receiver. The receiver uses 8× oversampling, which means that one UART bit is represented by eight receiver timing positions. Rather than depending on one sample, the design takes three samples around the middle of the bit. These samples are then evaluated using majority voting.

3. **Deserializer**

The deserializer converts the serial data recovered by the sampling block into an 8-bit parallel data word. Each received data bit is shifted into an internal register. After all eight bits have been received, the register represents the complete data byte.

Why is this block required?

The UART communication line carries one bit at a time, while the output interface requires an eight-bit parallel value. The deserializer performs this serial-to-parallel conversion.

4. **Start Check**

The start-check block verifies that the detected start bit is actually logic 0. Because the UART line normally remains at logic 1, the transition to logic 0 indicates the possible beginning of a frame. The receiver samples the expected start-bit position to confirm that the line is still low. If the sampled value is not zero, the receiver treats the event as an invalid start condition and returns to the idle state.

Why is this block required?

This check prevents the receiver from interpreting a short or incorrect transition as a valid UART frame.

5. **Parity Check**

The parity-check block verifies the parity bit received from the transmitter.

The receiver calculates the expected parity using the reconstructed data and the selected parity configuration. It then compares the calculated value with the received parity bit. If they do not match, a parity error is detected.

Why is this block required?

Parity checking provides a simple method of detecting corruption of the received frame.

6. **Stop Check**

The stop-check block verifies the final bit of the UART frame. According to the UART specification, the stop bit must be logic 1. If the receiver detects logic 0 instead, the frame is considered invalid.

7. **RX Finite State Machine**

The RX FSM controls the complete reception process. The receiver progresses through:

IDLE → START → DATA → PARITY → STOP → IDLE

In the IDLE state, the receiver waits for the serial line to transition from its idle value of 1 to 0. The START state verifies that the detected start bit is valid. The DATA state receives the eight data bits and enables the deserializer. If parity is enabled, the FSM then enters the PARITY state to check the received parity bit. Finally, the STOP state verifies the stop bit. If the frame is valid, the receiver asserts data_valid. If an error is detected, valid data is not reported.

Why is this block required?

The receiver must perform several operations in a precise sequence. The FSM coordinates the counters, sampler, deserializer, and error-checking blocks so that the entire frame is processed correctly.

Top Level System Integration

The final system integrates the UART transmitter and receiver into a single module. The transmitter serial output is internally connected to the receiver serial input. This creates a UART loopback configuration, allowing the data transmitted by the TX module to be immediately processed by the RX module. The same clock, reset, prescale value, and parity configuration are provided to both modules. The communication path is TX Parallel Input to UART Transmitter to TX Serial Output to Internal UART Communication Line to UART Receiver to RX Parallel Output. The top-level module also exposes the transmitter busy signal and receiver data valid signal, making it possible to monitor the communication process during simulation.

Simulation Results

This section should contain your Questa Sim waveform screenshots and simulation output.

1. **Test Even Parity**

Input Data: 10100101
Parity: Enabled
Parity Type: Even

The simulation demonstrates that the transmitter accepts the input data, becomes busy, and generates the appropriate UART frame. The receiver samples the serial signal and reconstructs the original data.

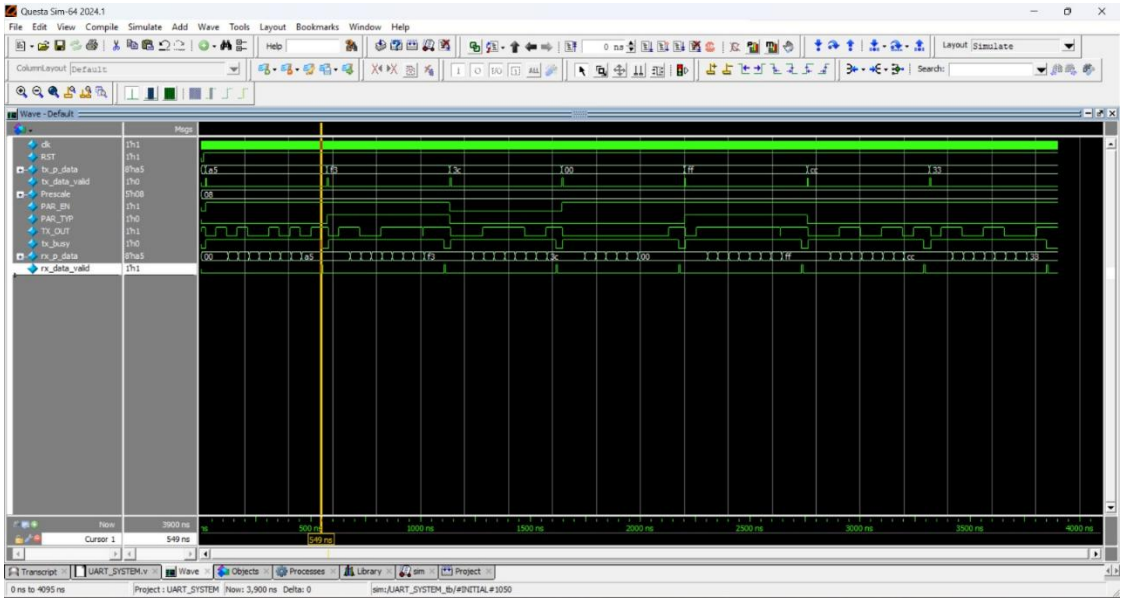

Figure 3: Test 1

2. Test Odd Parity

Input Data: 11110011

Parity: Enabled

Parity Type: Odd

This test verifies that the design correctly changes its parity behavior when odd parity is selected. The receiver successfully checks the received parity according to the odd-parity configuration and produces the original data.

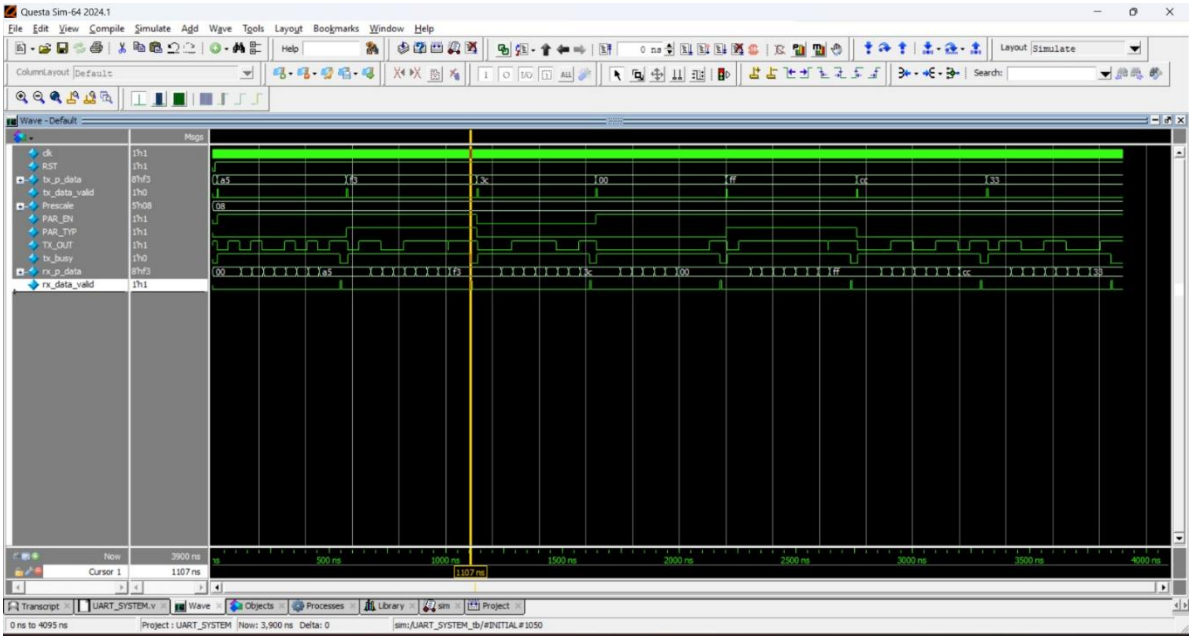

Figure 4: Test 2

3. Test No Parity

Input Data: 00111100

Parity: Disabled

This test verifies that the transmitter can generate a frame without a parity bit and that the receiver correctly skips the parity-checking stage. The received data must match the transmitted data.

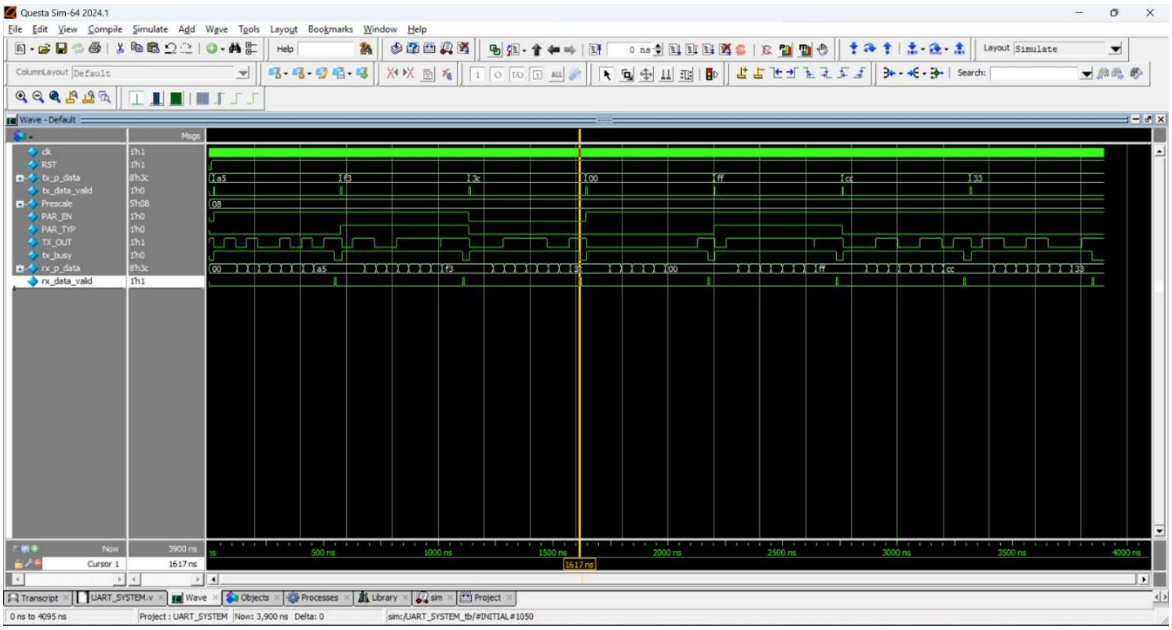

Figure 5: Test 3

4. Test All-Zero Data

Input Data: 00000000

Parity: Enabled

Parity Type: Even

This test verifies the design using an extreme data pattern containing only zeros. Testing such a pattern is useful because it ensures that the serializer, receiver sampling, and deserializer operate correctly even when all data bits have the same value.

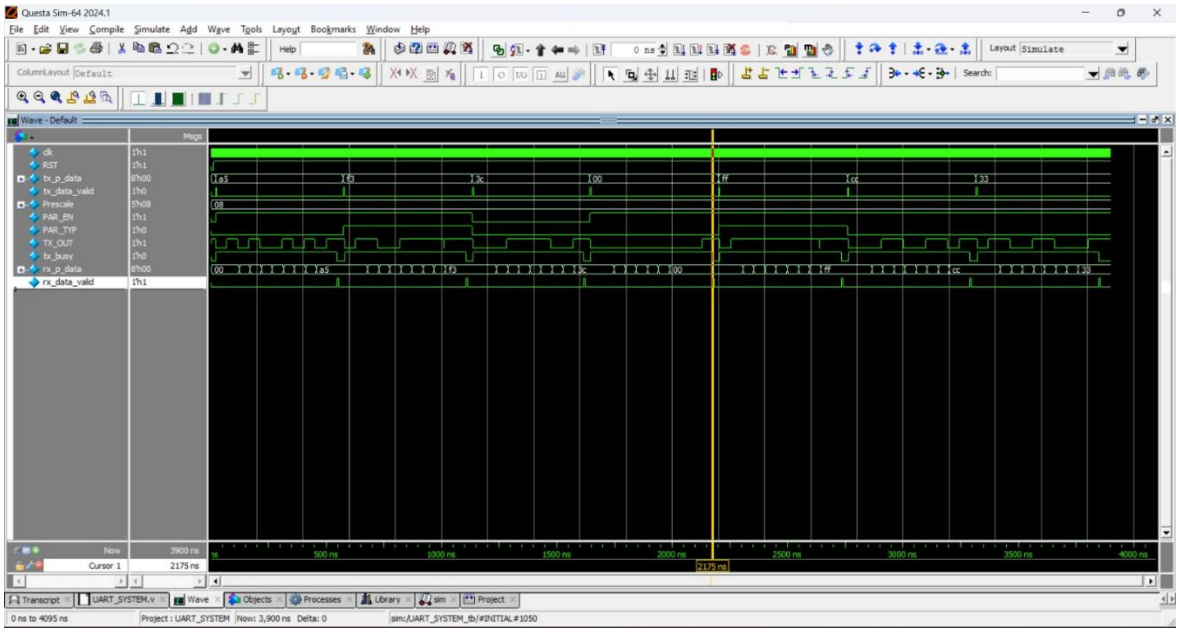

Figure 6: Test 4

5. Test All-One Data

Input Data: 11111111
Parity: Enabled
Parity Type: Odd

This test provides the opposite extreme by transmitting a byte containing only ones. The successful recovery of this pattern demonstrates that the design is not dependent on alternating or mixed data values.

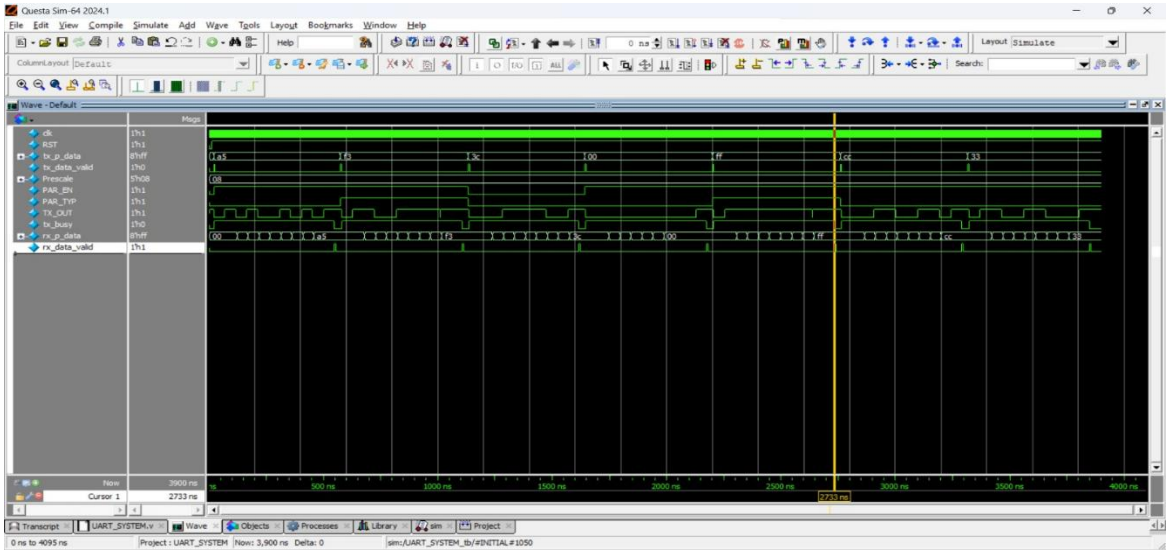

Figure 7: Test 5

Simulation Verification

The simulation results demonstrate that the developed UART system successfully performs the complete transmission and reception process. The transmitted data values are correctly reconstructed at the receiver output. The different parity configurations are handled correctly, and the receiver only asserts data valid after a valid frame has been processed.

The simulation also confirms the correct behavior of the transmitter output-selection blocks. The receiver architecture uses timing counters, majority-vote sampling, deserialization, and dedicated start, parity, and stop checking's busy signal. While a frame is being transmitted, the transmitter remains busy and does not accept another data word for processing.

The UART output remains at logic 1 when the transmitter is idle, and each transmitted frame begins with a logic-0 start bit and ends with a logic-1 stop bit.

The combination of 8x oversampling and three-sample majority voting allows the receiver to determine the value of each incoming bit using samples taken around the center of the bit period.

Conclusion

This project successfully implemented and verified a complete 8-bit full-duplex UART communication system using Verilog HDL. The design includes a UART transmitter capable of converting parallel data into a serial UART frame with configurable parity, together with a UART receiver capable of detecting, sampling, checking, and reconstructing the transmitted frame. The transmitter architecture uses dedicated timing, parity-generation, serialization, control, and output-selection blocks. The receiver architecture uses timing counters, majority-vote sampling, deserialization, and dedicated start, parity, and stop checking blocks controlled by a finite state machine.

The complete design was integrated into a top-level loopback system and verified in Questa Sim using a 200 MHz clock. Multiple test cases were performed, including even parity, odd parity, no parity, all-zero data, all-one data, and consecutive frames. The simulation results demonstrate that the system meets the main functional requirements: correct frame generation, correct data recovery, parity support, correct idle behavior, proper busy/data-valid control, and successful processing of consecutive UART frames.